Rate Limiter Analysis - Dakota Lending Platform

📋 Table of Contents

1. Overview
2. Algorithm Identification
3. File-wise Breakdown
4. Code Flow Explanation
5. Implementation Deep Dive
6. Comparison with Other Algorithms
7. Edge Cases and Limitations
8. Why This Algorithm?
9. Conclusion

🎯 Overview

Kya hai Rate Limiting?

Rate limiting ek technique hai jo control karta hai ki kitne requests ek user/client/service ek specific time window mein kar sakta hai. Yeh prevent karta hai:

- Server overload
- DDoS attacks
- Resource exhaustion
- Fair usage ensure karna

Is Codebase mein Implementation:

- **Directory:** `lending/common/ratelimiter/`
- **Total Files:** 5 Go files
- **Storage Backend:** Redis (distributed rate limiting)
- **Pattern:** Interface-based design for testability

🔍 Algorithm Identification

✅ Fixed Window Counter Algorithm

Is codebase mein **Fixed Window Counter** algorithm use ho raha hai. Yeh sabse simple aur efficient rate limiting algorithm hai.

Kaise Identify Kiya?

Analysis karne ke baad yeh key indicators mile:

1. **Window-based counting:**
    - `WindowSizeInSec` field hai Rule struct mein

- Har window ek fixed time period define karta hai

2. **Simple Counter Increment:**
   - Redis `INCR` command use ho raha hai
   - No sliding window, no token generation
3. **Expiry at Window Start:**
   - Jab first request aata hai (`count == 1`), tab Redis key ko `EXPIRE` set karte hain
   - Window end hone par key automatically delete ho jati hai
4. **Binary Decision:**
   - Agar count <= MaxReq → Request pass
   - Agar count > MaxReq → Request reject

Mathematical Model:

```
if (count(current_window) <= MaxReq) → Allow
else → Deny
```

📁 File-wise Breakdown

1️⃣ **ratelimiter.go** (Interface Definition)

```go
package ratelimiter

type RateLimiter interface {
    Register(string, Rule)      // Rule register karo key ke liye
    Pass(context.Context, string) bool  // Check karo ki request pass honi
chahiye
}
```

Purpose:

- Interface define karta hai (Go's best practice)
- Dependency injection ke liye useful
- Testing mein mock implementation use kar sakte hain

Design Pattern: Interface Segregation Principle (SOLID)

2️⃣ **error.go** (Error Definitions)

```go
var (
    RateLimitExceeded = errors.New("rate_limit_exceeded")
```

```
)
```

Purpose:

- Centralized error definition
- Clients ko meaningful error return kar sakte hain
- Error handling standardize ho jati hai

Usage: Jab rate limit exceed ho jaye tab ye error return kiya ja sakta hai (though currently boolean return ho raha hai)

3️⃣ **redis_ratelimiter.go** (Core Implementation)

Key Components:

a) Rule Structure

```go
type Rule struct {
    MaxReq          int64  // Maximum requests allowed
    WindowSizeInSec int64  // Time window in seconds
}
```

Example:

- MaxReq = 100
- WindowSizeInSec = 60
- Meaning: 100 requests per minute allowed

b) redisRateLimiter Struct

```go
type redisRateLimiter struct {
    rules map[string]Rule     // Key → Rule mapping
    redis redis.Client        // Redis connection
    stats statsd.Client       // Metrics/monitoring
}
```

Why Redis?

- Distributed systems mein kaam karta hai (multiple servers)
- Atomic operations (INCR, EXPIRE)
- Fast performance (in-memory)
- Built-in TTL support

c) Register Method

```
func (rl *redisRateLimiter) Register(key string, rule Rule) {
    if _, exists := rl.rules[key]; exists {
        panic(fmt.Sprintf("duplicated key prefix %s for ratelimiter",
key))
    }
    rl.rules[key] = rule
}
```

Flow:

1. Check karo ki key already registered hai ya nahi
2. Agar hai → Panic (duplicate keys not allowed)
3. Nahi hai → Rule store karo in-memory map mein

Example Registration:

```
rateLimiter.Register("loan_application", Rule{
    MaxReq: 50,
    WindowSizeInSec: 60,
})
// Meaning: loan_application service can handle 50 requests per minute
```

d) Pass Method (Main Logic) 🚀

```
func (rl *redisRateLimiter) Pass(ctx context.Context, key string) bool {
    // 1. Rule exists?
    rule, exists := rl.rules[key]
    if !exists {
        return true  // No limit = always pass
    }

    // 2. Increment counter in Redis
    count, err := rl.redis.Incr(ctx, key)
    if err != nil {
        slog.FromContext(ctx).Error(logTag, "Failed to increment...")
        return true  // Error = relax check (availability over strict
limiting)
    }

    // 3. Send metrics
    rl.stats.Histogram(logTag, "tps", float64(count), "key:"+key)

    // 4. Check if limit exceeded
    if count > rule.MaxReq {
        rl.stats.Count1(logTag, "limit_reached", "key:"+key)
        return false  // 🚫 Reject request
    }
```

```
    // 5. Set expiry on first request
    if count == 1 {
        _, err := rl.redis.Expire(ctx, key,
time.Duration(rule.WindowSizeInSec)*time.Second)
        if err != nil {
            slog.FromContext(ctx).Error(logTag, "Failed to add expire...")
        }
    }

    // 6. Final check
    return count <= rule.MaxReq  // ✅ Allow request
}
```

Critical Design Decisions:

1. **Fail-open Strategy:**
   - Agar Redis error aaye → `return true`
   - System availability > strict rate limiting
   - Production systems mein common practice
2. **Expiry on First Request:**
   - `count == 1` par hi `EXPIRE` set hota hai
   - Efficient approach (no repeated expiry calls)
3. **Monitoring Integration:**
   - StatsD metrics send karte hain
   - Real-time monitoring possible

4️⃣ **redis_ratelimiter_test.go** (Unit Tests)

Test Cases:

Test Case	MaxReq	Count	Expected	Reason
Rule doesn't exist	2	1	✅ Pass	No rule = no limit
Below limit	2	1	✅ Pass	1 < 2
At limit	2	2	✅ Pass	2 == 2
Exceeds limit	2	3	❌ Fail	3 > 2

Test Structure:

```
mockRedis.On("Incr", mock.Anything, "asset_offline").Return(tt.increments,
nil)
mockRedis.On("Expire", mock.Anything, "asset_offline",
time.Duration(10)*time.Second).Return(true, nil)
```

```
result := rateLimiter.Pass(context.Background(), tt.key)
assert.Equal(t, tt.expected, result)
```

5️⃣ **mock_rate_limiter.go** (Generated Mock)

Purpose:

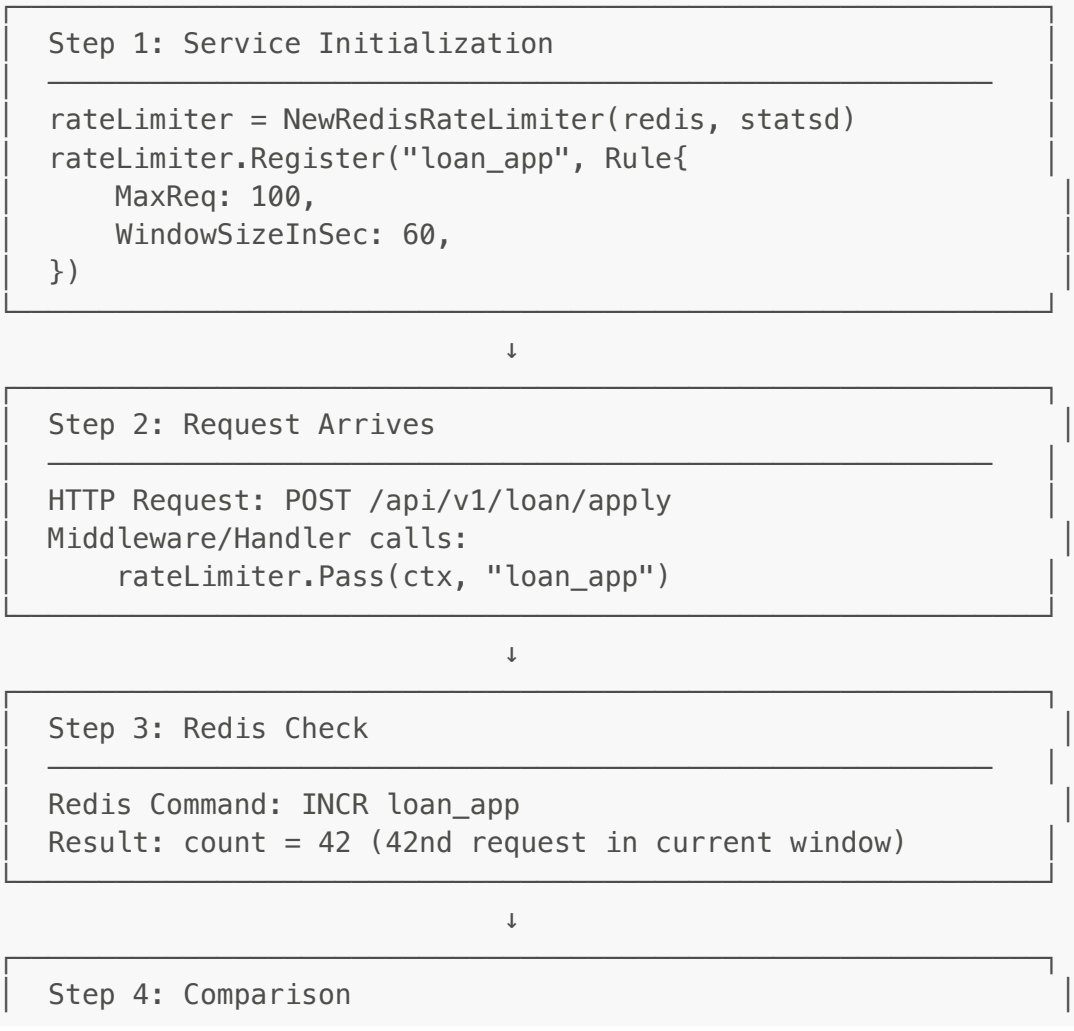
- Auto-generated by `mockery` tool
- Testing mein use hota hai
- Actual Redis calls avoid karne ke liye

Command:

```
//go:generate mockery --name RateLimiter --inpackage --case=underscore
```

🔄 Code Flow Explanation

Scenario: User sends a loan application request



```
│  ──────────────────────────────────────────────────────────────  │
│  Is 42 <= 100? → Yes                                            │
│  Decision: ALLOW REQUEST                                        │
└─────────────────────────────────────────────────────────────────┘
                                ↓
┌─────────────────────────────────────────────────────────────────┐
│  Step 5: Metrics                                                │
│  ──────────────────────────────────────────────────────────────  │
│  stats.Histogram("ratelimiter", "tps", 42, "key:loan_app")      │
└─────────────────────────────────────────────────────────────────┘
                                ↓
┌─────────────────────────────────────────────────────────────────┐
│  Step 6: Response                                               │
│  ──────────────────────────────────────────────────────────────  │
│  return true → Request proceeds to business logic               │
└─────────────────────────────────────────────────────────────────┘
```

Timeline Visualization

```
Time:  0s          10s         20s         30s         40s         50s
60s
       |           |           |           |           |           |
|
       ├──────────────────────────── Window 1 (60s)
──────────────────────────────┤
       |
|
Req 1  ●  count=1  (INCR + EXPIRE 60s)
|
Req 2     ●  count=2
|
Req 3        ●  count=3
|
...
|
Req 100                                                  ●
count=100    |
Req 101                                                     ● ❌
REJECTED|

|
       ├──────────────────────────── Window 2 (60s)
──────────────────────────────┤
       | (Key expired, counter reset to 0)
|
Req 1  ●  count=1  (INCR + EXPIRE 60s)
|
```

🔬 Implementation Deep Dive

Redis Operations

1. INCR Command:

```
INCR loan_app
# Result: 1 (if key doesn't exist)
# Result: 2 (if previous value was 1)
```

Why INCR?

- Atomic operation (race condition nahi hota)
- Multiple servers simultaneously use kar sakte hain
- Returns the incremented value

2. EXPIRE Command:

```
EXPIRE loan_app 60
# Key will be deleted after 60 seconds
```

Why only when count == 1?

- Efficiency: Har request par EXPIRE call karna wasteful
- First request creates the window
- Redis automatically deletes key after TTL

Real-world Example

Scenario: Asset Offline Service

- **Registered Rule:** 50 requests per 10 seconds
- **Current Time:** 14:30:00

Time	Request	Redis Count	Decision	Reason
14:30:00.100	Req-1	1	✅ Pass	First in window, EXPIRE set to 10s
14:30:01.200	Req-2	2	✅ Pass	Within limit
14:30:05.500	Req-25	25	✅ Pass	Within limit
14:30:08.800	Req-50	50	✅ Pass	At limit boundary
14:30:09.900	Req-51	51	❌ Reject	**Exceeded!**
14:30:10.100	Req-52	1	✅ Pass	**New window!** Key expired & reset

---

⚖️ Comparison with Other Algorithms

1. Token Bucket

How it works:

- Tokens ek fixed rate se bucket mein aate hain
- Har request ek token consume karta hai
- Bucket full ho jaye to tokens drop ho jaate hain

Difference from Fixed Window:

- Token Bucket allows "bursts" (jaldi-jaldi requests)
- More complex implementation
- Better for traffic smoothing

Why not used here?

- Fixed Window simpler hai
- Redis mein implement karna complex hota
- Current use case ke liye overkill

---

2. Leaky Bucket

How it works:

- Requests ek queue mein jate hain
- Queue se requests constant rate se process hote hain

Difference from Fixed Window:

- Leaky Bucket smoothens traffic
- Fixed rate guarantee
- Queue management required

Why not used here?

- Memory overhead (queue maintenance)
- Latency badh sakta hai (queuing delay)
- Complexity high

---

3. Sliding Window Log

How it works:

- Har request ki timestamp store hoti hai
- Purāne timestamps (window se bahar) remove hote hain
- Count karte hain active timestamps

Difference from Fixed Window:

- No "boundary problem"
- Accurate rate limiting
- More fair distribution

Why not used here?

- Memory expensive (har timestamp store karna)
- Redis mein list/set operations slow
- Overkill for most use cases

---

4. Sliding Window Counter

How it works:

- Previous + Current window ka weighted average
- More accurate than Fixed Window
- Less memory than Sliding Window Log

Difference from Fixed Window:

- Smoothens boundary issues
- Better approximation of true rate
- Slightly complex calculation

Why not used here?

- Fixed Window sufficient hai current needs ke liye
- Implementation complexity badh jata
- Performance impact (multiple Redis calls)

---

5. Fixed Window Counter (CURRENT) ✅

Why Chosen:

✅ **Simple Implementation:**

- Just one Redis key per rule
- INCR + EXPIRE, that's it!

✅ **High Performance:**

- O(1) time complexity
- Minimal Redis operations
- Low memory footprint

✅ **Distributed System Ready:**

- Redis ensures consistency across servers
- No need for coordination between instances

✅ **Easy to Understand:**

- Junior developers easily understand
- Maintainable code

✅ **Good Enough for Most Cases:**

- Banking/fintech applications mein acceptable
- Boundary issues rare hain

⚠ Edge Cases and Limitations

1. **Boundary Problem** (Classic Fixed Window Issue)

Problem:

```
Window 1: [0s - 60s]    Window 2: [60s - 120s]
MaxReq: 100 per minute

Time: 50s → 100 requests (all allowed) ✅
Time: 70s → 100 requests (all allowed) ✅

Actual rate in [50s - 70s]: 200 requests in 20 seconds!
= 600 requests/minute!! 🚨
```

Impact:

- Short bursts mein double requests aa sakti hain
- True rate limiting violation

Mitigation in this code:

- Monitoring via StatsD helps detect anomalies
- MaxReq should be set conservatively

---

2. **Redis Failure Handling**

Code:

```go
if err != nil {
    return true  // Relax check on error
}
```

Implication:

- Agar Redis down hai → All requests pass
- Availability > Strict limiting
- Temporary period mein over-consumption possible

Alternative Approach:

- Could return `false` (strict mode)

- Circuit breaker pattern
- Fallback to in-memory limiter

Why current approach is good:

- Production systems mein service uptime critical
- Rate limiting secondary concern (comparatively)
- Better user experience

---

3. **Race Conditions?**

Q: Kya multiple servers se simultaneously requests aane par problem hogi?

A: ❌ NO - Redis INCR is atomic!

```
Server 1: INCR loan_app → 42
Server 2: INCR loan_app → 43  (not 42 again!)
Server 3: INCR loan_app → 44
```

Redis guarantees atomicity, so no race condition.

---

4. **Memory Considerations**

Current Approach:

- Each registered rule: ~50 bytes in-memory map
- Each active key in Redis: ~100 bytes

Scale:

- 1000 rules = ~50 KB memory
- 10000 active keys in Redis = ~1 MB

Verdict: Very efficient! ✅

---

5. **Clock Skew**

Problem:

- Agar multiple servers ka time sync nahi hai?

Solution:

- Redis acts as single source of truth
- All TTL calculations Redis par hote hain
- Server clocks don't matter

🤔 Why This Algorithm?

Business Context: Lending Platform

Requirements:

1. **High Throughput:** Thousands of loan applications per minute
2. **Low Latency:** Users ko fast response chahiye
3. **Fair Usage:** Prevent abuse, DDoS
4. **Distributed System:** Multiple microservices

Why Fixed Window Counter Wins:

Requirement	Fixed Window	Other Algorithms
Performance	⚡ Excellent (O(1))	Token/Leaky: Good, Sliding: Slower
Simplicity	✅ Very Simple	❌ Complex
Memory	💾 Low	📦 High (especially Sliding Log)
Accuracy	⚠ Good Enough	✅ Better (Sliding variants)
Distributed	✅ Redis native	⚠ Needs coordination

Trade-offs Accepted:

❌ **Sacrifice:** Perfect accuracy (boundary problem) ✅ **Gain:** Speed, simplicity, low resource usage

Verdict: Right choice for this use case! 🎯

🎓 Conclusion

Summary

1. **Algorithm Used: Fixed Window Counter** with Redis backend
2. **Implementation Quality:**
   - ✅ Clean interface design
   - ✅ Fail-open strategy for resilience
   - ✅ Good test coverage
   - ✅ Monitoring integrated
   - ✅ Production-ready
3. **Files Overview:**
   - `ratelimiter.go`: Interface
   - `redis_ratelimiter.go`: Core logic
   - `error.go`: Error definitions
   - `redis_ratelimiter_test.go`: Unit tests
   - `mock_rate_limiter.go`: Test mocks

4. **Key Strengths:**
    - Simple aur efficient
    - Distributed systems ke liye perfect
    - Low latency, high throughput
    - Easy to maintain
5. **Known Limitations:**
    - Boundary problem (acceptable trade-off)
    - Redis dependency (mitigated by fail-open)

Real-world Usage

Is rate limiter ka use ho raha hai in services:

- `asset-offline`: Asset management operations
- `perfios-adapter`: Third-party integration
- `cr-decision-eng`: Credit decision engine
- `cb-my-adapter`: Credit bureau adapter

Recommendation

Kya improvement kar sakte hain?

1. **Add Sliding Window Counter:**
    - For critical endpoints
    - Better accuracy chahiye jahan
2. **Circuit Breaker Integration:**
    - Redis failures ke liye better handling
3. **Dynamic Rule Updates:**
    - Runtime mein rules change karne ka option
4. **Better Error Responses:**
    - Client ko batana chahiye "retry after X seconds"
5. **Rate Limit Headers:**
    - HTTP headers mein remaining quota show karna
    - Example: `X-RateLimit-Remaining: 42`

Overall Rating: ⭐⭐⭐⭐ (4/5)

Excellent implementation for a production lending platform. Simple, efficient, aur effective! 🚀

📚 Additional Resources

Learn More:

- Redis INCR: https://redis.io/commands/incr
- Rate Limiting Algorithms: https://cloud.google.com/architecture/rate-limiting-strategies-techniques
- Distributed Systems Patterns: https://martinfowler.com/articles/patterns-of-distributed-systems/

Monitoring Queries (if using StatsD/Datadog):

```
# TPS per service
sum:ratelimiter.tps{*} by {key}

# Rate limit hits
sum:ratelimiter.limit_reached{*} by {key}

# Alert if > 100 rate limit hits per minute
sum(last_1m):sum:ratelimiter.limit_reached{*} > 100
```

Document Created: February 5, 2026
Author: Senior Backend Engineer Analysis
Version: 1.0
Codebase: Dakota Lending Platform